The four-colour Rado number of $2(x-y)=3z$ is 226

A cube-and-conquer computation with checked DRAT proofs

Takumi Ichikawa
github.com/IchiTaku326

22 October 2026
(not peer reviewed; computation finished 22 October 2026 JST)

Abstract

For a linear equation E and $k \geq 1$, the Rado number $\mathrm{R}_k(E)$ is the least n such that every k-colouring of $\{1,\dots,n\}$ contains a monochromatic solution of E. Chang, De Loera and Wesley [1] computed many such numbers by SAT solving; for the equation $2(x-y)=3z$ and $k=4$ colours they report only the lower bound $\mathrm{R}_4 > 225$. We show that $\mathrm{R}_4(2(x-y)=3z)=226$. The lower bound is an explicit 4-colouring of $\{1,\dots,225\}$, checked by an independent program. The upper bound is a cube-and-conquer proof: 14,162 cubes, each shown unsatisfiable, with every cube re-solved by CaDiCaL and its DRAT proof accepted by `drat-trim`. All programs, logs and the colouring are public.

1 Introduction

Rado's theorem [2] characterises the linear homogeneous equations that are *regular*, i.e. that have a monochromatic solution in every finite colouring of the positive integers. For a regular equation E the Rado number $\mathrm{R}_k(E)$ is finite for every k. For the family $a(x-y)=bz$ with $\gcd(a,b)=1$ the paper [1] proves, among other results, $\mathrm{R}_3(a(x-y)=bz)=a^3$ for $a \geq b+2$, gives the lower bound $\mathrm{R}_k \geq a^k$ for all k, and tabulates computed values for 2, 3 and (for small a,b) 4 colours. In their four-colour table the entry for $a=2$, $b=3$ is ">225"; to our knowledge no exact value has been published (Section 7). This note closes that entry.

Theorem 1. $\mathrm{R}_4(2(x-y)=3z)=226$.

Contributions. The new content is (i) the exact value in Theorem 1, (ii) a complete, resumable pipeline whose every step leaves a time-stamped log, and (iii) a DRAT certificate for each of the 14,162 cases, regenerable in about one hour on a desktop PC.

Conventions. Here a *solution* is a triple (x,y,z) of positive integers with $2(x-y)=3z$; the variables need not be distinct (for example $(4,1,2)$ and $(5,2,2)$ are solutions). Throughout, "colouring of $[n]$" means a map $[n]=\{1,\dots,n\} \to \{0,1,2,3\}$, and it is *good* if it has no monochromatic solution with $x,y,z \leq n$. Then $\mathrm{R}_4 > n$ iff a good colouring of $[n]$ exists, and Theorem 1 is the conjunction of the two statements below.

Theorem 2 (Lower bound)**.** *There is a good* 4*-colouring of* $[225]$.

Theorem 3 (Upper bound)**.** *There is no good* 4*-colouring of* $[226]$.

2 Lower bound

A good colouring of [225] was found by a SAT solver (CaDiCaL through PySAT) on 20 October 2026 (JST) and is stored in `results/lower_bound_n225/`. It is checked by `tools/rado_verify.py`, a program written to be as plain as possible: three nested loops over $x, y, z \le n$ and a test of $2(x-y) = 3z$ and equality of the three colours; it does not share code with the CNF generator. The check reports `VALID`. A good colouring of [225] restricts to a good colouring of every $[m]$, $m \le 225$, so $\mathrm{R}_4 > m$ for all $m \le 225$.

3 Upper bound

3.1 Encoding

For $n = 226$, $k = 4$ we use Boolean variables $v_{j,c}$ ($j \in [n]$, $c \in \{0,1,2,3\}$): "j has colour c". The clauses are:

1. each j gets at least one colour, and at most one colour (6 binary clauses per j);
2. for every *solution set* $S = \{x, y, z\}$ and every colour c the clause $\bigvee_{j \in S} \neg v_{j,c}$ (the set S may have two elements if $z = x$ or $z = y$); there are 8,400 solution sets in [226];
3. symmetry breaking (colours numbered in order of first use): auxiliary variables $u_{j,c}$ meaning "colour c occurs in $1..j$" ($c = 0, 1, 2$) with the defining clauses, together with $v_{1,c} \to \bot$ for $c \ge 1$ and $v_{j,c} \to u_{j-1,c-1}$ for $c \ge 1$.

The formula has 1,582 variables and 37,891 clauses. Every good colouring can be renamed so that colours first appear in the order $0, 1, 2, 3$ (restricted growth); renaming preserves goodness, and the renamed colouring satisfies the formula. Hence the formula is satisfiable iff a good colouring of [226] exists.

3.2 Cubes

Let $d = 10$. A *cube* is a restricted-growth string of length d over $\{0,1,2,3\}$ (first letter 0; each letter at most one larger than the maximum so far) with no monochromatic solution inside $[d]$. There are exactly 14,162 cubes; this number was obtained by the search program and re-computed by an independent brute-force enumeration of all 4^{10} strings.

Lemma 1. *If a good colouring of* [226] *exists, then the formula together with the unit clauses of some cube is satisfiable.*

Proof. Rename the colours of a good colouring into restricted-growth form. Its restriction to [10] is a restricted-growth string without monochromatic solution, i.e. one of the cubes, and the colouring satisfies the formula by the previous subsection. □

3.3 Cube search

Each of the 14,162 cubes was solved by Glucose (via PySAT, assumptions, conflict budget $2 \cdot 10^4$, budgets multiplied by 4 on retry) in parallel with 11 workers; all returned UNSAT (21 October 2026, 05:38–09:26 JST, about 41 CPU-hours in total including retried attempts).

3.4 Certification

To avoid trusting an incremental solver, in a second pass (22 October 2026, 01:15–02:23 JST, about 12 CPU-hours) each cube was solved again by CaDiCaL as a separate CNF (formula plus unit clauses), producing a DRAT proof, and `drat-trim` [5] accepted *every* proof: `VERIFIED 14162/14162`. The proofs total about 28 GB (largest: 38 MB) and are therefore not stored; the second pass regenerates them. Lemma 1 and the 14,162 UNSAT certificates give Theorem 3.

4 Computational environment

All computations ran on one desktop PC.

CPU	Intel Core i5-11400F (6 cores, 12 threads)
Memory	32 GB DDR4
GPU	NVIDIA GeForce RTX 3060 (not used; CDCL solvers run on the CPU)
System	Windows; proof checking in WSL (Linux)
Workers	11 processes for the cube search and for the proof check
Lower bound	CaDiCaL 1.5.3 through PySAT
Cube search	Glucose 4 through PySAT, assumptions, budget $2 \cdot 10^4$ conflicts
Proof check	CaDiCaL 3.0.1 (static build) and `drat-trim` (version not recorded)

5 Summary of the computation

Variables / clauses of the formula ($n = 226$)	1,582 / 37,891
Solution sets of $2(x - y) = 3z$ in [226]	8,400
Cubes (depth 10)	14,162
Cubes UNSAT / SAT	14,162 / 0
DRAT proofs accepted by `drat-trim`	14,162 / 14,162
Total size of the proofs	about 28 GB
Cube search, wall clock	3 h 48 min (11 workers)
Proof check, wall clock	about 68 min (11 workers)

6 What this result depends on

We list the places where an error could still hide.

1. **Encoding.** The formula generator is the only place where the problem statement enters the proof. Evidence for its correctness: agreement with exhaustive search on small cases; reproduction of published values (Schur numbers, $\mathrm{R}_4(x - y = 2z) = 171$ and the exact 3-colour values of [1] in the range we tested; 338 comparisons with published values or theorems in a larger table, no disagreement); and validity of the lower-bound colouring under the independent checker.
2. **Symmetry breaking.** The argument above is a pen-and-paper argument; it was not machine-checked.
3. **Cube list.** Completeness rests on Lemma 1 and the count 14,162.
4. **Proof checker.** `drat-trim` is widely used but is itself unverified software; a check with a formally verified checker such as `cake_lpr` [6] has not been done.
5. **Hardware and software.** Runs were done on a single desktop PC (11 worker processes); the version of `drat-trim` was not recorded.

7 Literature status

We checked, on 22 October 2026, the arXiv listing of [1] (only version v1, 7 October 2022, whose four-colour table gives > 225 for $a = 2, b = 3$), a student report from a SAT group on Rado numbers (three colours only), and arXiv:2505.12085 (three colours, other equations); none states a value for this equation. This search was not exhaustive: the list of papers citing [1] and the authors have not yet been consulted. If the value is already known, this note should be regarded as an independent verification.

8 Reproducibility

Code, logs and the colouring: https://github.com/IchiTaku326/rado-226 (MIT licence); archived copy (all versions): https://doi.org/10.5281/zenodo.23565747. Step 0 validates the colouring (seconds), step 1 reruns the cube search (hours on about 11 cores, resumable), step 2 reruns the DRAT check (about one hour on 11 cores). Every row of `results/` carries the JST time at which it was written.

Acknowledgements and disclosure

The programs, the computation plan and parts of the text were developed with the help of an AI assistant (Claude, Anthropic). The author is responsible for the content. This is a small computational result; it has no bearing on the P versus NP problem.

References

[1] Y. Chang, J. A. De Loera, W. J. Wesley, *Rado numbers and SAT computations*, arXiv:2210.03262 (2022), DOI 10.1145/3476446.3535494.

[2] R. Rado, Studien zur Kombinatorik, *Math. Z.* **36** (1933), 424–470.

[3] M. J. H. Heule, O. Kullmann, V. W. Marek, Solving and verifying the Boolean Pythagorean triples problem via cube-and-conquer, *SAT 2016*, LNCS 9710, 228–245.

[4] M. J. H. Heule, Schur number five, *Proc. AAAI 2018*; arXiv:1711.08076.

[5] N. Wetzler, M. J. H. Heule, W. A. Hunt Jr., DRAT-trim: efficient checking and trimming using expressive clausal proofs, *SAT 2014*, LNCS 8561, 422–429.

[6] Y. K. Tan, M. J. H. Heule, M. Myreen, cake_lpr: verified propagation redundancy checking in CakeML, *TACAS 2021*, LNCS 12652.

[7] A. Biere, K. Fazekas, M. Fleury, M. Heisinger, CaDiCaL, Kissat, Paracooba, Plingeling and Treengeling entering the SAT Competition 2020, *SAT Competition 2020: Solver and Benchmark Descriptions*, 51–53.

[8] G. Audemard, L. Simon, Predicting learnt clauses quality in modern SAT solvers, *IJCAI 2009*, 399–404.

[9] A. Ignatiev, A. Morgado, J. Marques-Silva, PySAT: a Python toolkit for prototyping with SAT oracles, *SAT 2018*, LNCS 10929, 428–437.

[10] B. Landman, A. Robertson, *Ramsey Theory on the Integers*, 2nd ed., AMS, 2014.